DESARROLLO DE INTERFACES



U07 - Proyecto entregable 02 – Codificando con Dart -

Miriam Mendieta Racedo.-
2DAMS
24/02/2026

ÍNDICE

INTRODUCCIÓN **1**

KATA_1.DART 2
KATA_2.DART 4
KATA_3.DART 5
KATA_4.DART 7
KATA_5.DART 9
KATA_6.DART 10
KATA_7.DART 11
KATA_8.DART 12
KATA_9.DART 13
KATA_10.DART 14
KATA_11.DART 15
KATA_12.DART 17
KATA_13.DART 18
KATA_14.DART 20
KATA_15.DART 21

BIBLIOGRAFÍA **22**

INTRODUCCIÓN

Dart es un lenguaje de programación de código abierto, lanzado por Google en 2011, diseñado para crear aplicaciones eficientes tanto en web como en dispositivos móviles. Su uso se ha popularizado especialmente gracias a Flutter, el framework que permite desarrollar apps nativas para Android, iOS, web y escritorio desde una sola base de código.

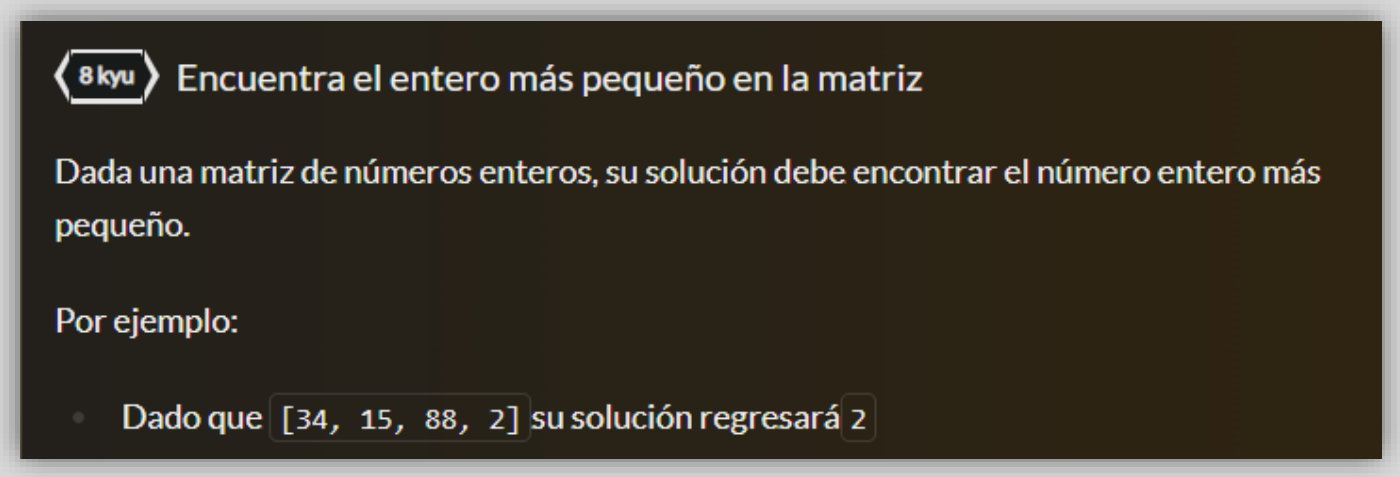
Características principales:

- Orientado a objetos: todo en Dart es un objeto, lo que facilita la modularidad y reutilización del código.
- Sintaxis clara y moderna: similar a C#, Java y JavaScript, lo que lo hace accesible para quienes ya conocen otros lenguajes.
- Tipado estático con inferencia: permite detectar errores en tiempo de compilación sin necesidad de declarar tipos explícitos.
- Compilación flexible: puede compilarse a JavaScript para web o a código nativo para móviles y escritorio.
- Soporte para programación funcional: incluye funciones de primer orden, expresiones lambda y manejo elegante de colecciones.
- Gestión de errores robusta: con try-catch, throw y tipos nulos controlados desde Dart 2.12 (null safety).
- Documentación integrada: permite generar documentación profesional directamente desde el código con comentarios tipo docstring.



U07 - Proyecto entregable 02 - Codificando con Dart -

kata_1.dart



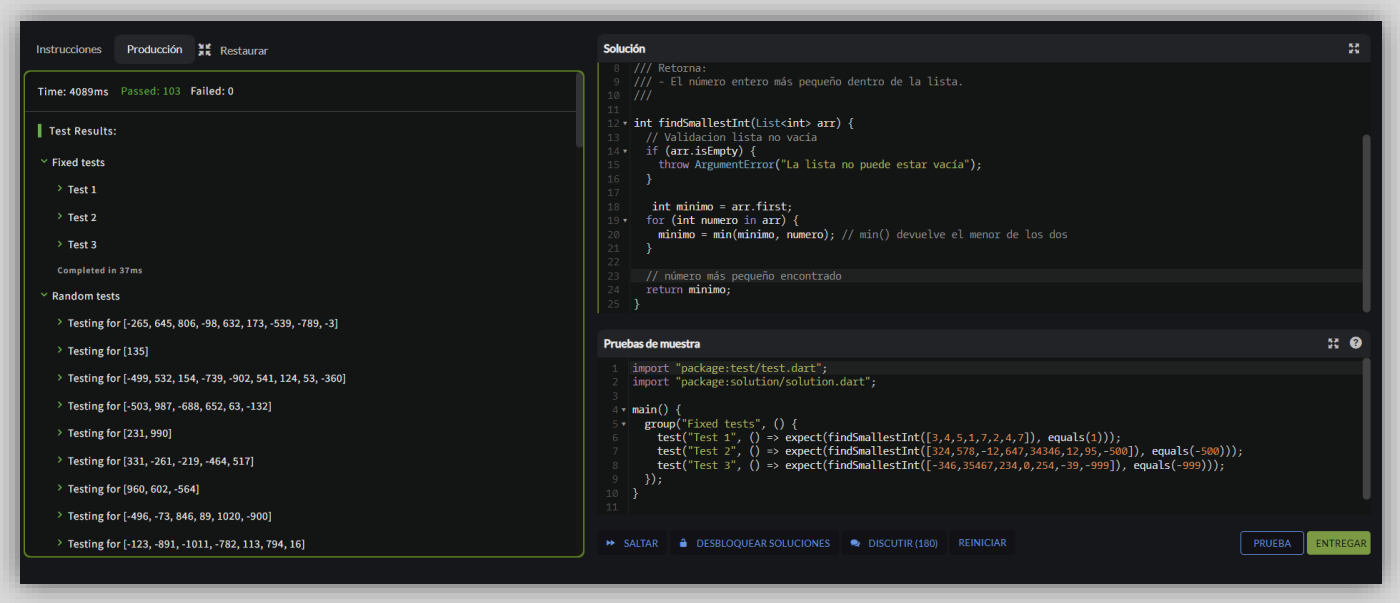
Dada una matriz de números enteros, su solución debe encontrar el número entero más pequeño.

Por ejemplo:

Dado que [34, 15, 88, 2]su solución regresará2

Dado que [34, -345, -1, 100]su solución regresará-345

Se puede asumir, para el propósito de este kata, que la matriz suministrada no estará vacía.



```dart
import 'dart:math'; // Importar math para la función min()

/// Encuentra el entero más pequeño en una lista de enteros.
///
/// Parámetros:
///         [arr]: Lista de enteros
///
/// Retorna:
///         El número entero más pequeño dentro de la lista.
///

int findSmallestInt(List<int> arr) {
  // Validar  lista no vacía
  if (arr.isEmpty) {
    throw ArgumentError("La lista no puede estar vacía");
  }

  //  mínimo  primer elemento
  int minimo = arr.first;

  // Recorrer la lista y comparar cada número con el mínimo actual
  for (int numero in arr) {
    minimo = min(minimo, numero); // min() devuelve el menor de los dos
  }

  //  número más pequeño encontrado
  return minimo;
}

void main() {
  List<int> ejemplo = [34, 15, 88, 2];
  print("El número más pequeño es: ${findSmallestInt(ejemplo)}");
}
```

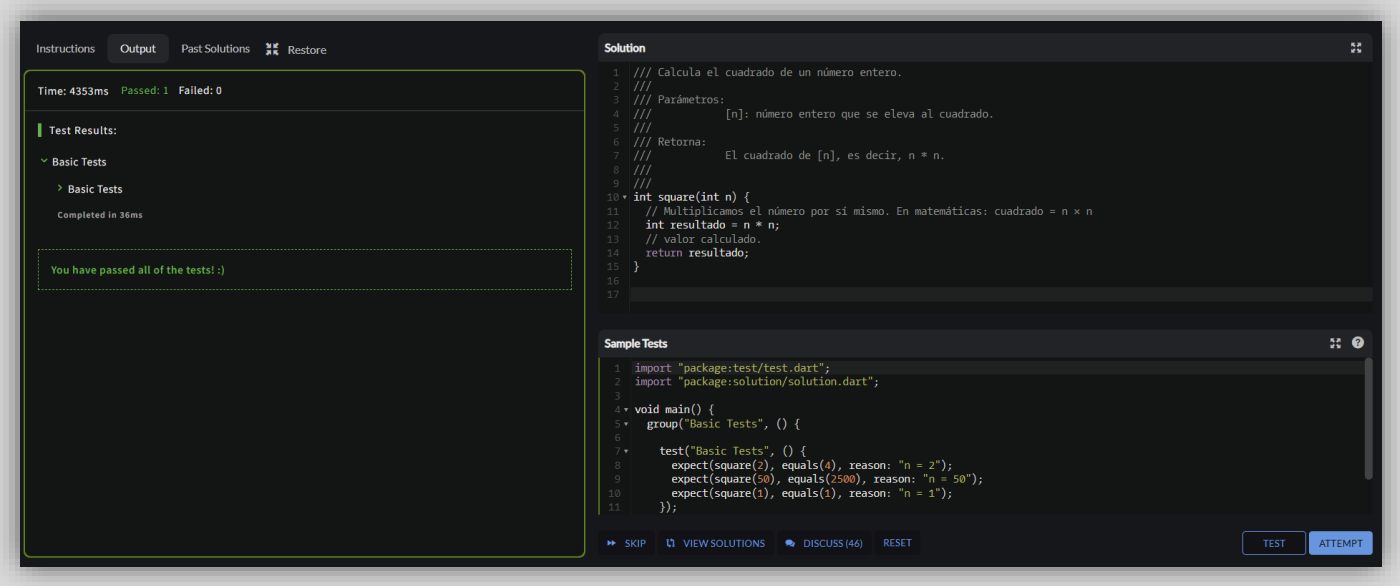
```
El número más pequeño es: 2
```

U07 - Proyecto entregable 02 - Codificando con Dart -

kata_2.dart

8 kyu Función 2: elevar al cuadrado un argumento

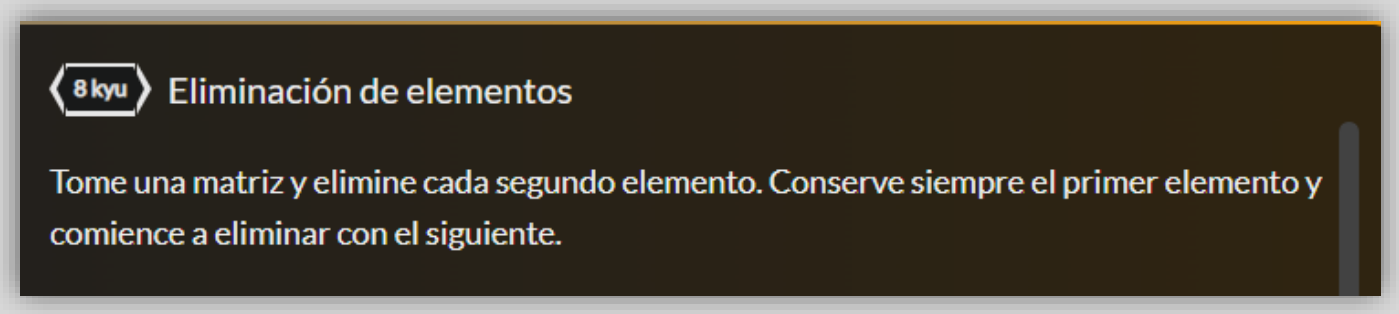
Ahora tienes que escribir una función que tome un argumento y devuelva el cuadrado del mismo.



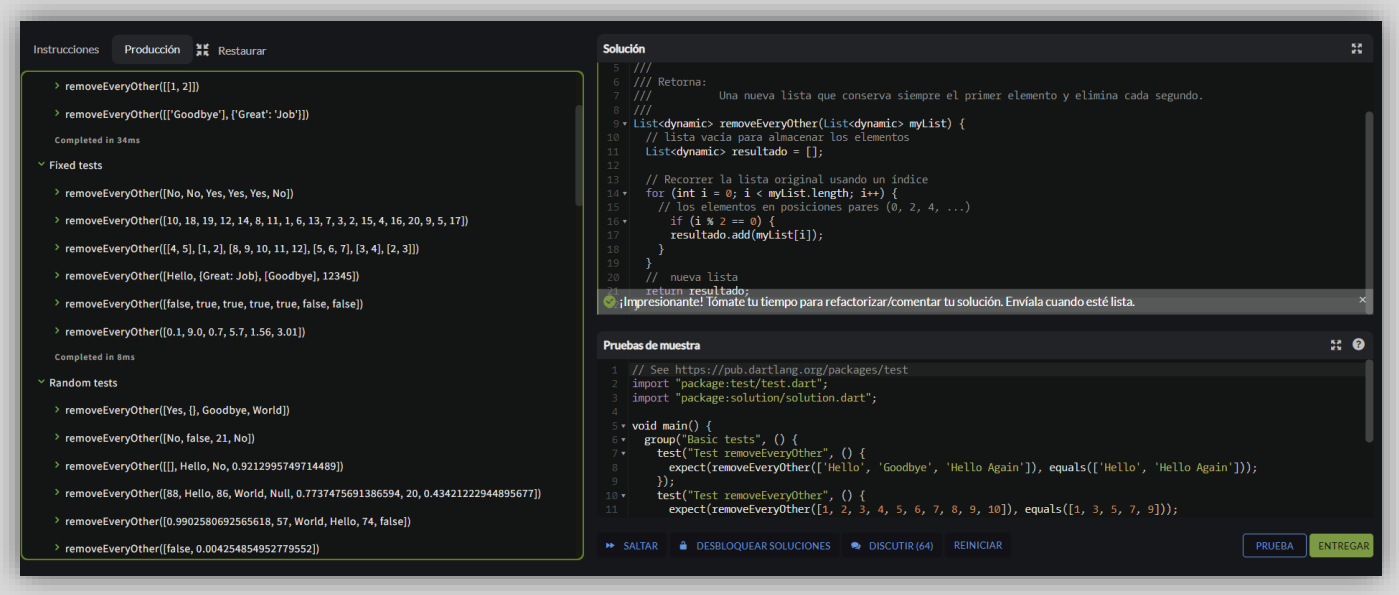

```dart
/// Calcula el cuadrado de un número entero.
/// Parámetros:
///         [n]: número entero que se eleva al cuadrado.
/// Retorna:
///         El cuadrado de [n], es decir, n * n.
///
int square(int n) {
  // Multiplicamos el número por sí mismo. En matemáticas: cuadrado = n × n
  int resultado = n * n;
  // valor calculado.
  return resultado;
}

void main() {
  print(square(0));  // 0 → el cuadrado de 0 es 0
  print(square(2));  // 4 → el cuadrado de 2 es 4
  print(square(-3)); // 9 → el cuadrado de -3 es 9
}
```

kata_3.dart



Tome una matriz y elimine cada segundo elemento. Conserve siempre el primer elemento y comience a eliminar con el siguiente.
Ejemplo:
["Keep", "Remove", "Keep", "Remove", "Keep", ...]-->["Keep", "Keep", "Keep", ...]
¡Ninguna de las matrices estará vacía, por lo que no tendrás que preocuparte por eso!



```dart
/// Elimina cada segundo elemento de una lista.
///
/// Parámetros:
///         [myList]: Lista de elementos (puede contener cualquier tipo: String, int, etc.).
///
/// Retorna:
///         Una nueva lista que conserva siempre el primer elemento y elimina cada segundo.
///
List<dynamic> removeEveryOther(List<dynamic> myList) {
  // lista vacía para almacenar los elementos
  List<dynamic> resultado = [ ];

  // Recorrer la lista original usando un índice
  for (int i = 0; i < myList.length; i++) {
    // los elementos en posiciones pares (0, 2, 4, ...)
     if (i % 2 == 0) {
     resultado.add(myList[i]);
    }
  }
  //  nueva lista
  return resultado;
}

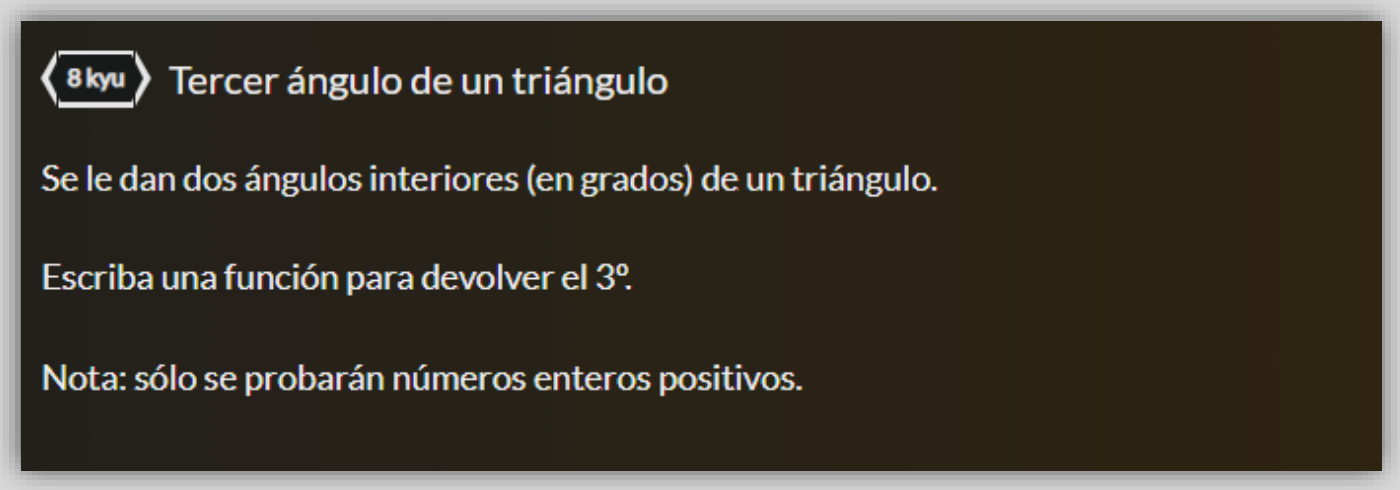
void main() {
  print(removeEveryOther(["Keep", "Remove", "Keep", "Remove", "Keep"]));
  // ["Keep", "Keep", "Keep"]

  print(removeEveryOther([1, 2, 3, 4, 5, 6]));
  // [1, 3, 5]

  print(removeEveryOther(["A", "B", "C", "D"]));
  // ["A", "C"]
}
```

```
[Keep, Keep, Keep]
[1, 3, 5]
[A, C]
```

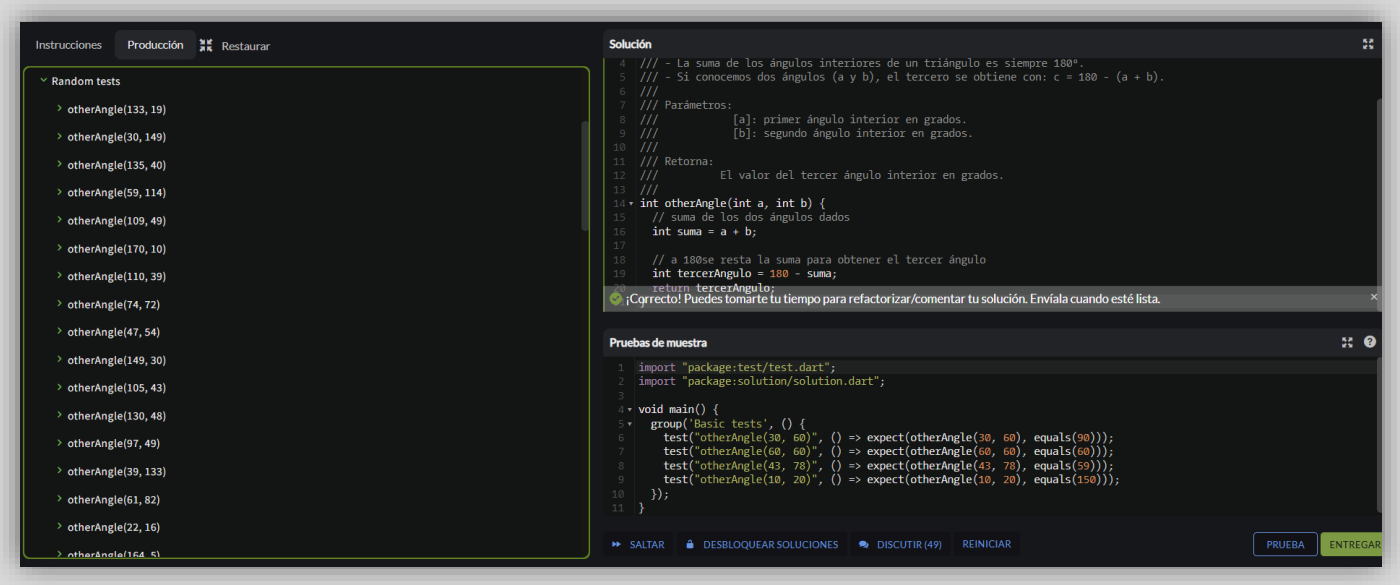
kata_4.dart



https://en.wikipedia.org/wiki/Triangle

En cualquier triángulo, la suma de los ángulos interiores es siempre 180°.

Si conocemos dos ángulos (a y b), el tercero (c) se calcula como: $c = 180 - (a+b)$



```dart
/// Calcula el tercer ángulo interior de un triángulo.
///
/// Fundamento:
/// - La suma de los ángulos interiores de un triángulo es siempre 180°.
/// - Si conocemos dos ángulos (a y b), el tercero se obtiene con: c = 180 - (a + b).
///
/// Parámetros:
///         [a]: primer ángulo interior en grados.
///         [b]: segundo ángulo interior en grados.
///
/// Retorna:
///        El valor del tercer ángulo interior en grados.
///
int otherAngle(int a, int b) {
 // suma de los dos ángulos dados
 int suma = a + b;

  // a 180se resta la suma para obtener el tercer ángulo
  int tercerAngulo = 180 - suma;
  return tercerAngulo;
}

void main() {
  // Probamos la función con distintos valores
  print(otherAngle(60, 60));   // 60 → triángulo equilátero
  print(otherAngle(90, 45));   // 45 → triángulo rectángulo
  print(otherAngle(100, 30));  // 50 → triángulo escaleno
}

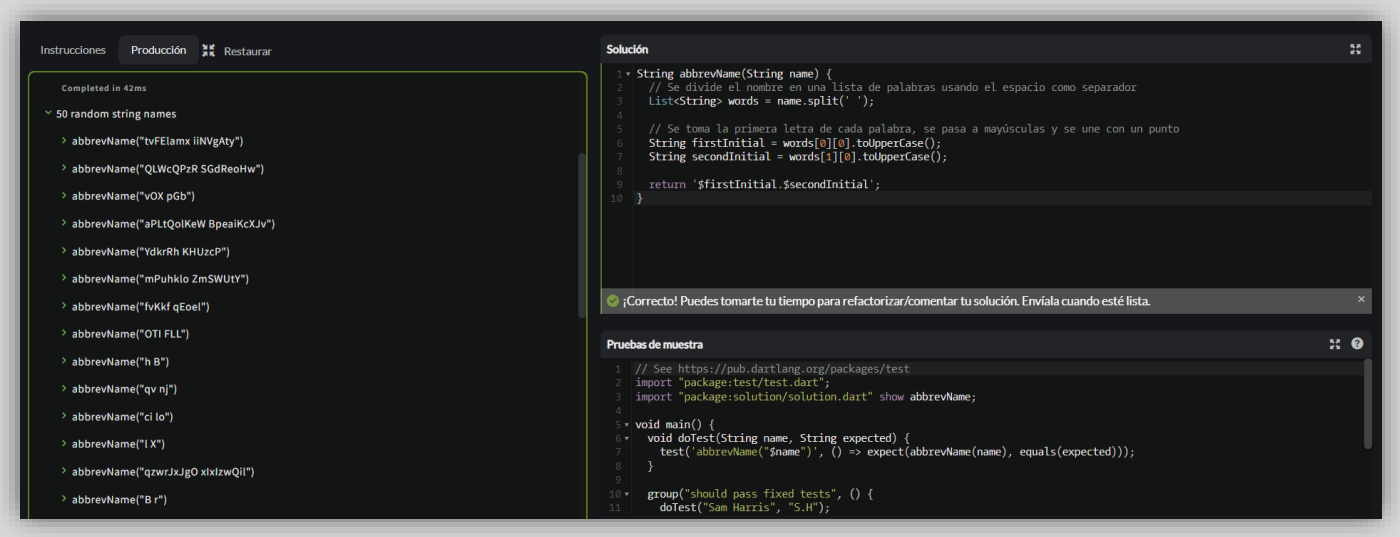
60
45
50
```

kata_5.dart

8 kyu Abreviar un nombre de dos palabras

Escribe una función para convertir un nombre en iniciales. Esta kata acepta estrictamente dos palabras con un espacio entre ellas.

El resultado debe ser dos letras mayúsculas separadas por un punto.




```dart
String abbrevName(String name) {
  // Se divide el nombre en una lista de palabras usando el espacio como separador
  List<String> words = name.split(' ');

  // Se toma la primera letra de cada palabra, se pasa a mayúsculas y se une con un punto
  String firstInitial = words[0][0].toUpperCase();
  String secondInitial = words[1][0].toUpperCase();

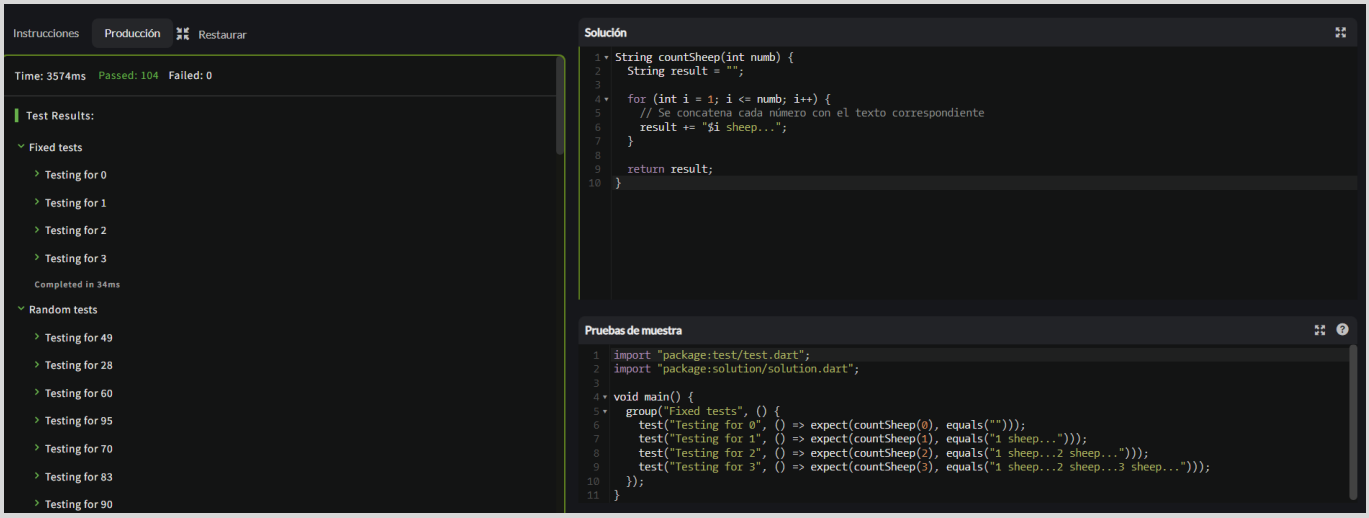
  return '$firstInitial.$secondInitial';
}
```

kata_6.dart

8 kyu Si no puedes dormir, ¡simplemente cuenta ovejas!

Tarea:

Dado un entero no negativo, `3` por ejemplo, devuelve una cadena con un murmullo: `"1 sheep...2 sheep...3 sheep..."`. La entrada siempre será válida; no se permiten enteros negativos.




```dart
String countSheep(int numb) {
  String result = "";

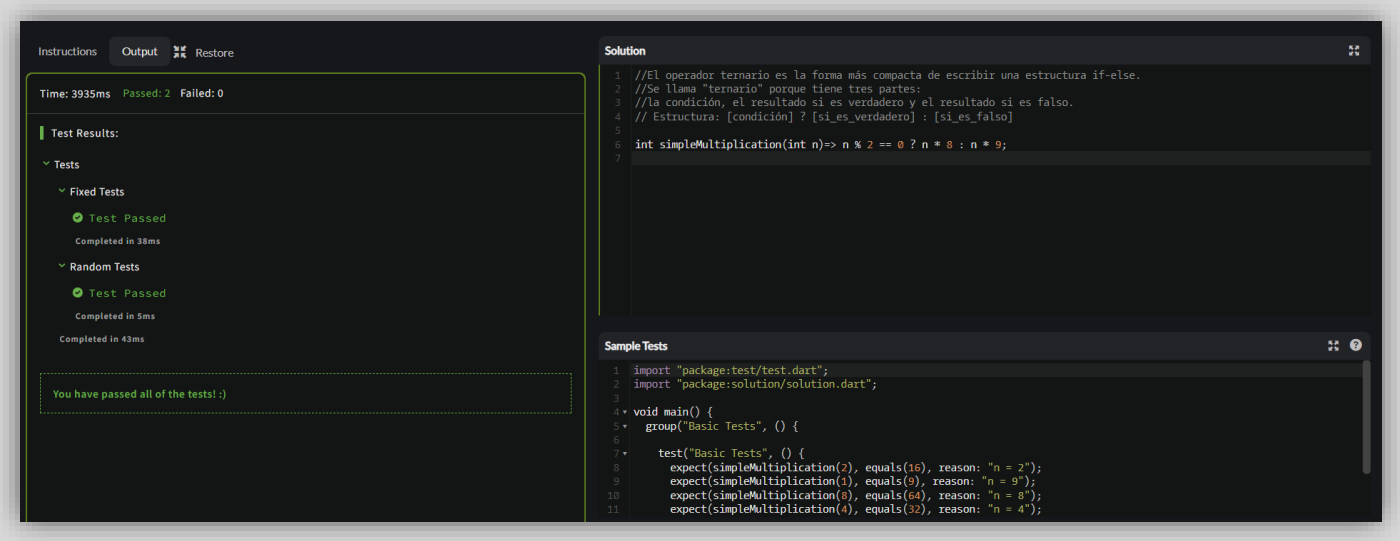
  for (int i = 1; i <= numb; i++) {
    // Se concatena cada número con el texto correspondiente
    result += "$i sheep...";
  }

  return result;
}
```

kata_7.dart

8 kyu Multiplicación simple

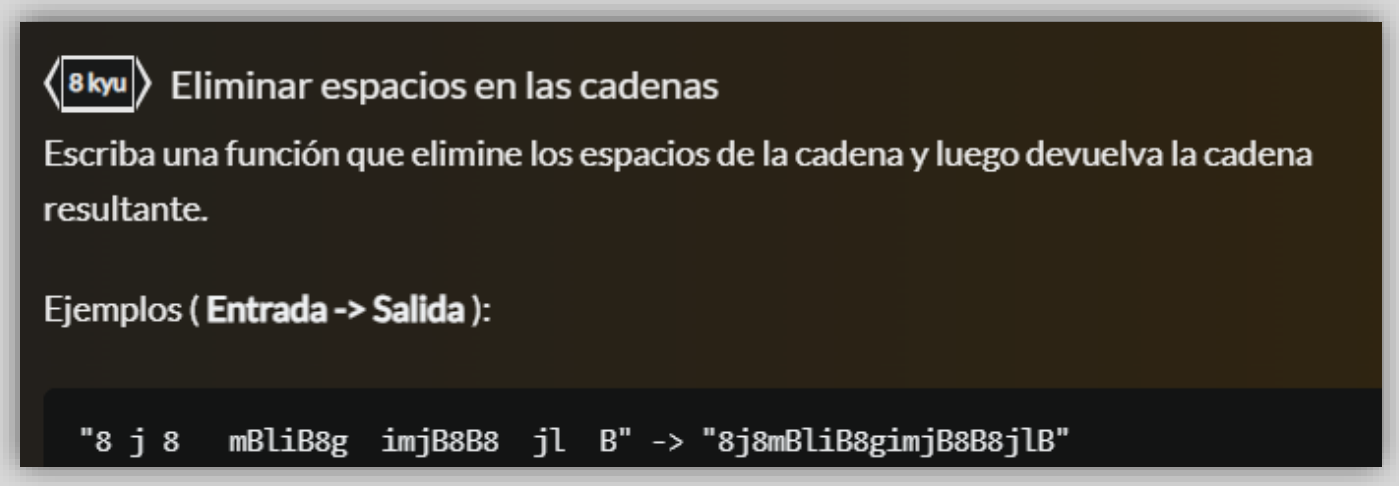
Este kata trata de multiplicar un número dado por ocho si es un número par y por nueve en caso contrario.

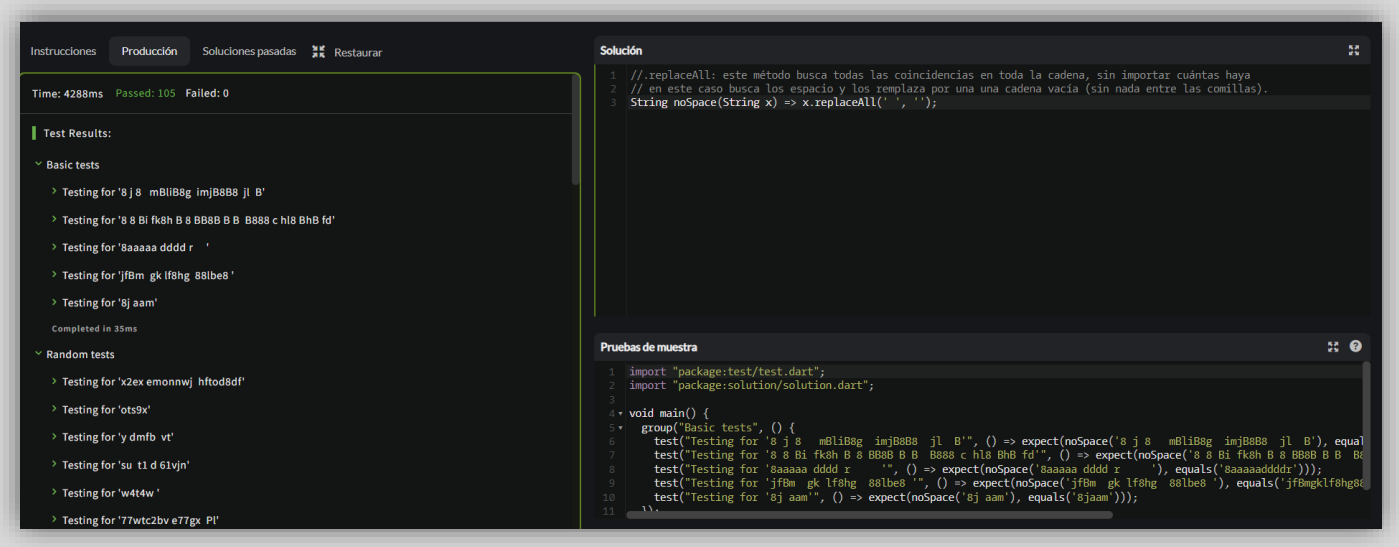



```dart
//El operador ternario es la forma más compacta de escribir una estructura if-else.
//Se llama "ternario" porque tiene tres partes:
//la condición, el resultado si es verdadero y el resultado si es falso.
// Estructura: [condición] ? [si_es_verdadero] : [si_es_falso]

int simpleMultiplication(int n)=> n % 2 == 0 ? n * 8 : n * 9;
```

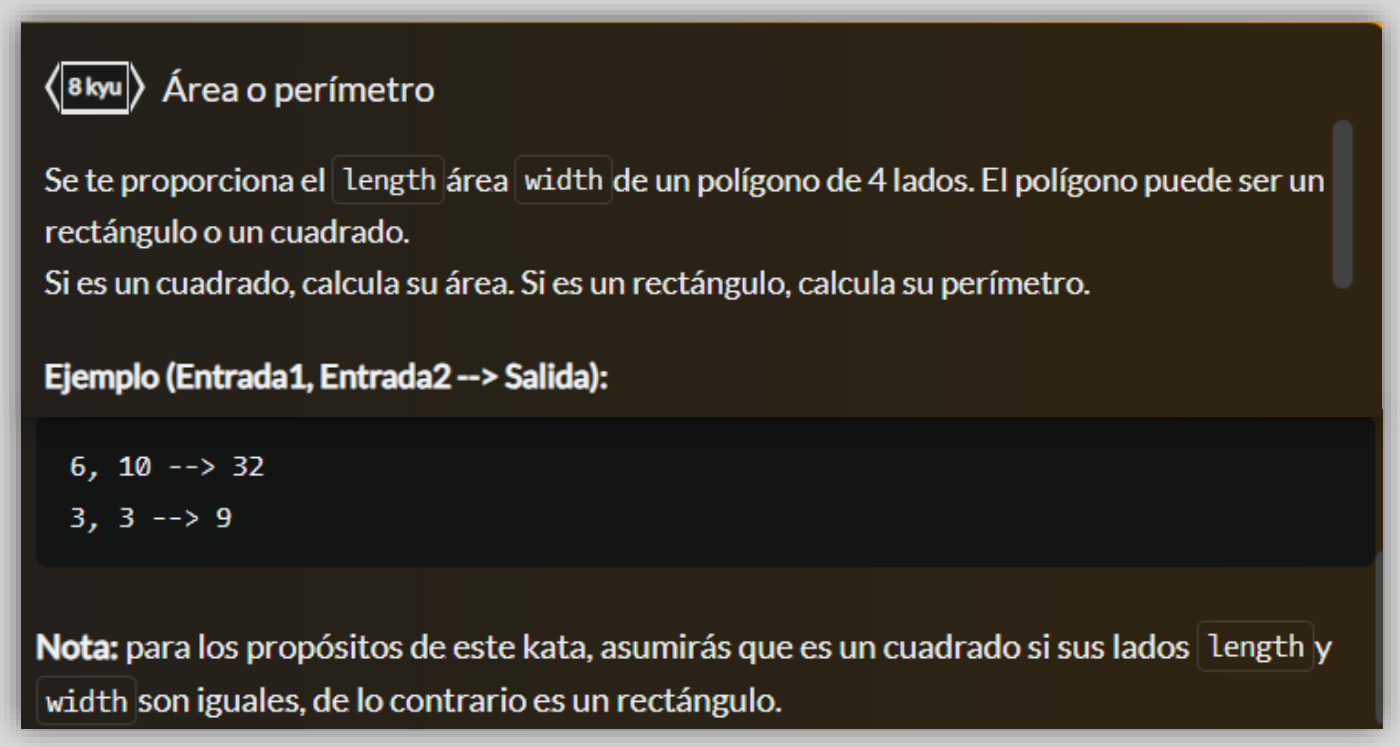
kata_8.dart






```dart
//.replaceAll: este método busca todas las coincidencias en toda la cadena, sin importar cuántas haya
// en este caso busca los espacio y los remplaza por una cadena vacía (sin nada entre las comillas).
String noSpace(String x) => x.replaceAll(' ', '');
```

kata_9.dart



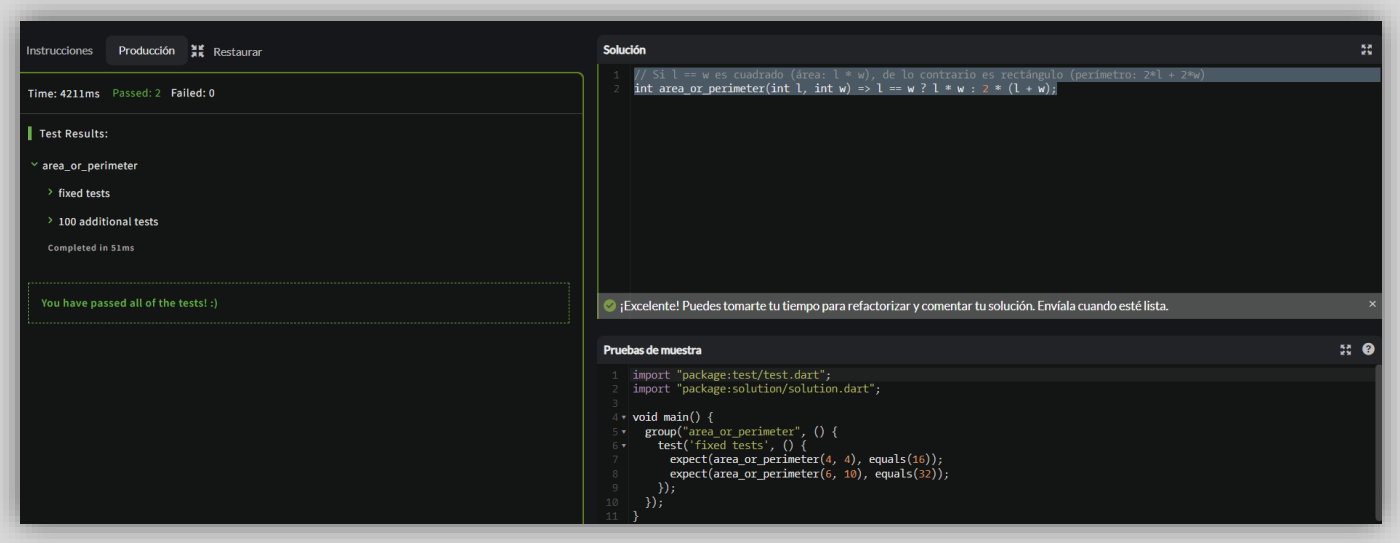
8 kyu Área o perímetro

Se te proporciona el `length` área `width` de un polígono de 4 lados. El polígono puede ser un rectángulo o un cuadrado.
Si es un cuadrado, calcula su área. Si es un rectángulo, calcula su perímetro.

Ejemplo (Entrada1, Entrada2 --> Salida):

```
6, 10 --> 32
3, 3 --> 9
```

Nota: para los propósitos de este kata, asumirás que es un cuadrado si sus lados `length` y `width` son iguales, de lo contrario es un rectángulo.

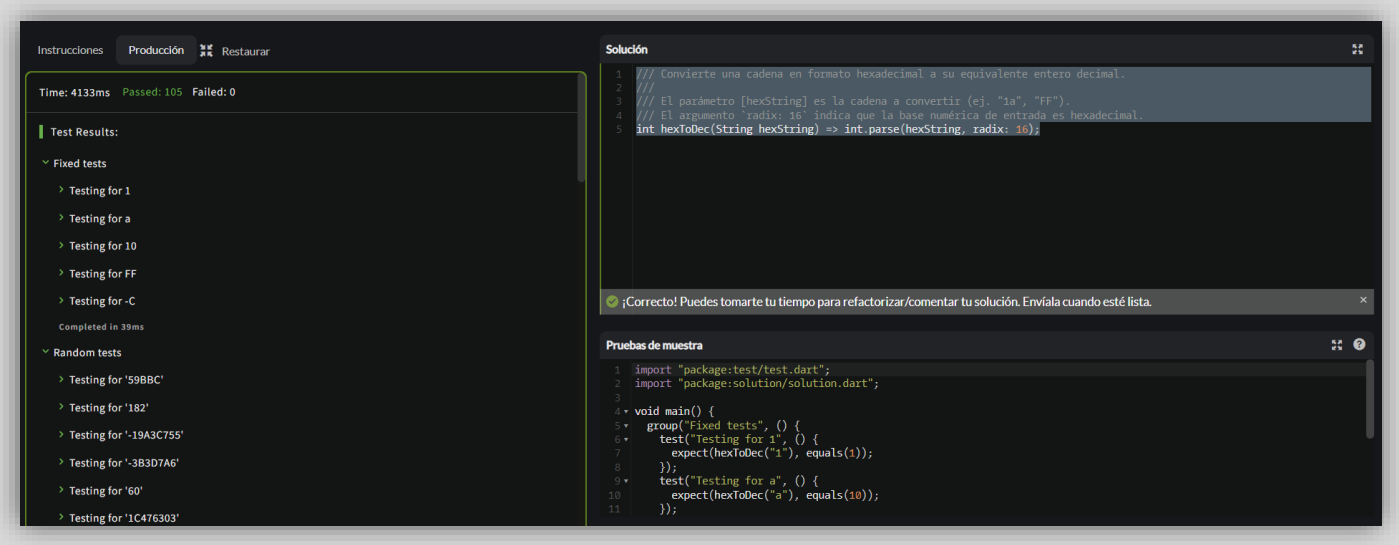



```dart
// Si l == w es cuadrado (área: l * w), de lo contrario es rectángulo (perímetro: 2*l + 2*w)
int area_or_perimeter(int l, int w) => l == w ? l * w : 2 * (l + w);
```

kata_10.dart

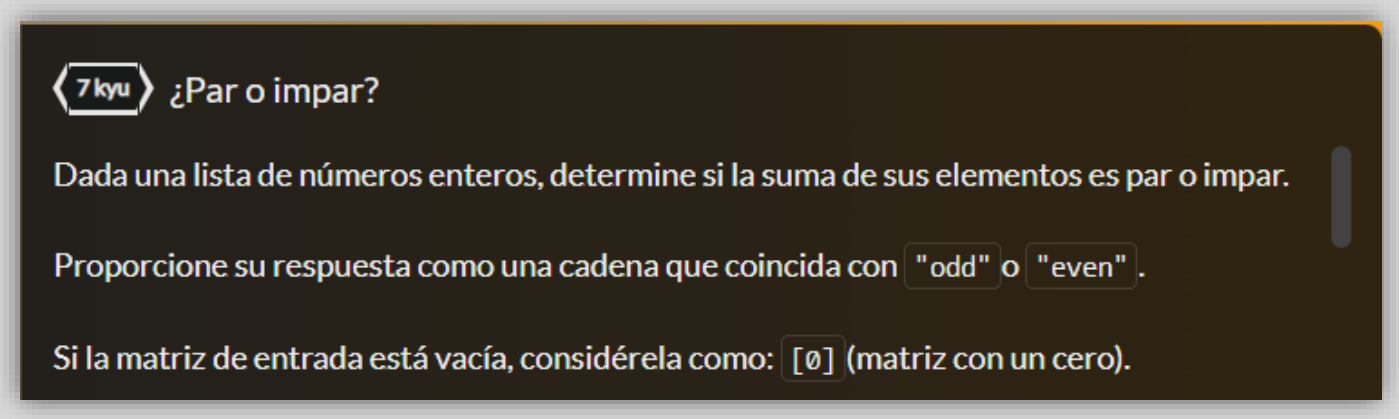
8 kyu Hexadecimal a decimal

Completa la función que convierte un número hexadecimal (dado como una cadena) en un número decimal.




```dart
/// Convierte una cadena en formato hexadecimal a su equivalente entero decimal.
///
/// El parámetro [hexString] es la cadena a convertir (ej. "1a", "FF").
/// El argumento `radix: 16` indica que la base numérica de entrada es hexadecimal.
int hexToDec(String hexString) => int.parse(hexString, radix: 16);
```

kata_11.dart

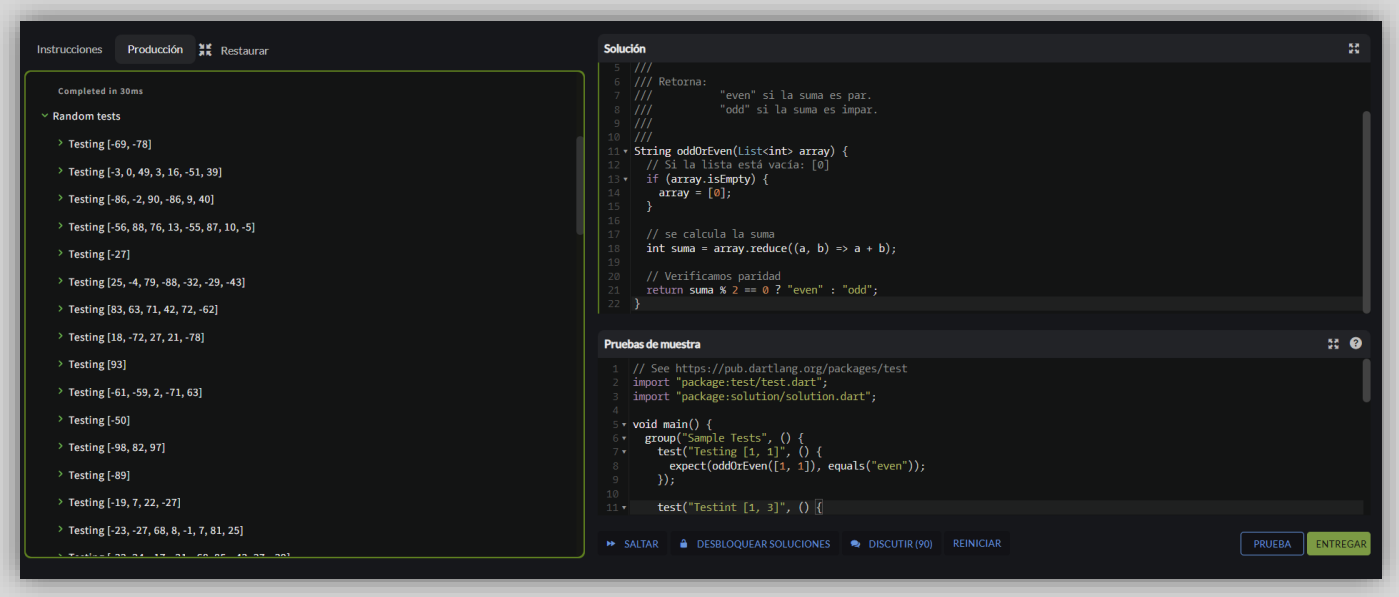


Tarea:
Dada una lista de números enteros, determine si la suma de sus elementos es par o impar.
Proporcione su respuesta como una cadena que coincida con "odd"o "even".
Si la matriz de entrada está vacía, considérela como: [0](matriz con un cero).

Ejemplos:
Input: [0]
Output: "even"

Input: [0, 1, 4]
Output: "odd"

Input: [0, -1, -5]
Output: "even"
¡Divertirse!



```dart
/// Determina si la suma de los elementos de la lista es par o impar.
///
/// Parámetros:
///         [array]: Lista de enteros.
///
/// Retorna:
///         "even" si la suma es par.
///         "odd" si la suma es impar.
///
///
String oddOrEven(List<int> array) {
  // Si la lista está vacía: [0]
  if (array.isEmpty) {
    array = [0];
  }

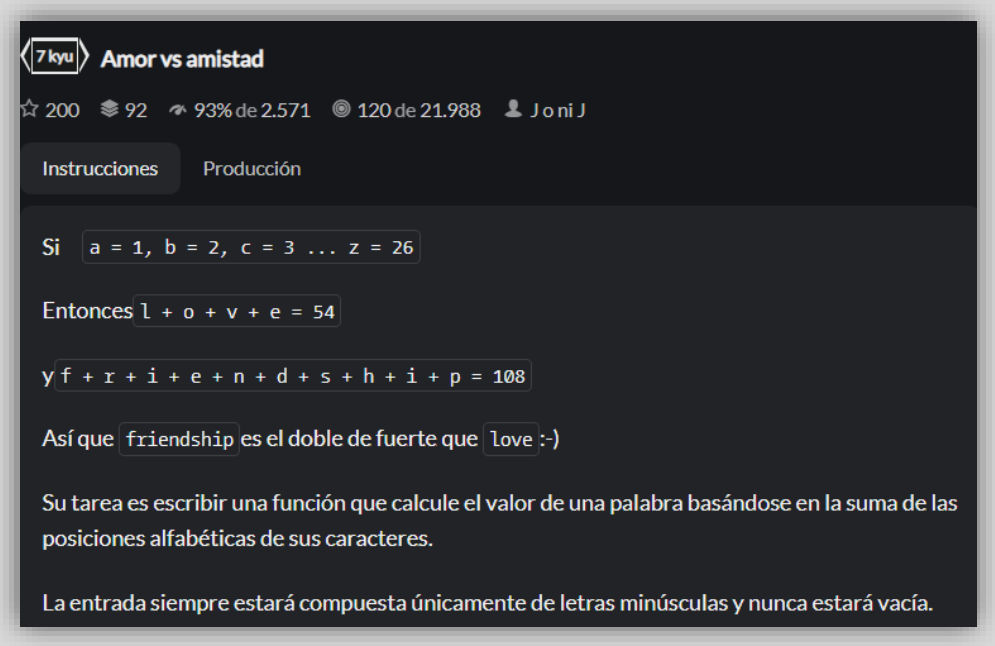
  // se calcula la suma
  int suma = array.reduce((a, b) => a + b);

  return suma % 2 == 0 ? "even" : "odd";
}

void main() {
  print(oddOrEven([0]));
  print(oddOrEven([0, 1, 4]));
  print(oddOrEven([0, -1, -5]));
  print(oddOrEven([]));
}

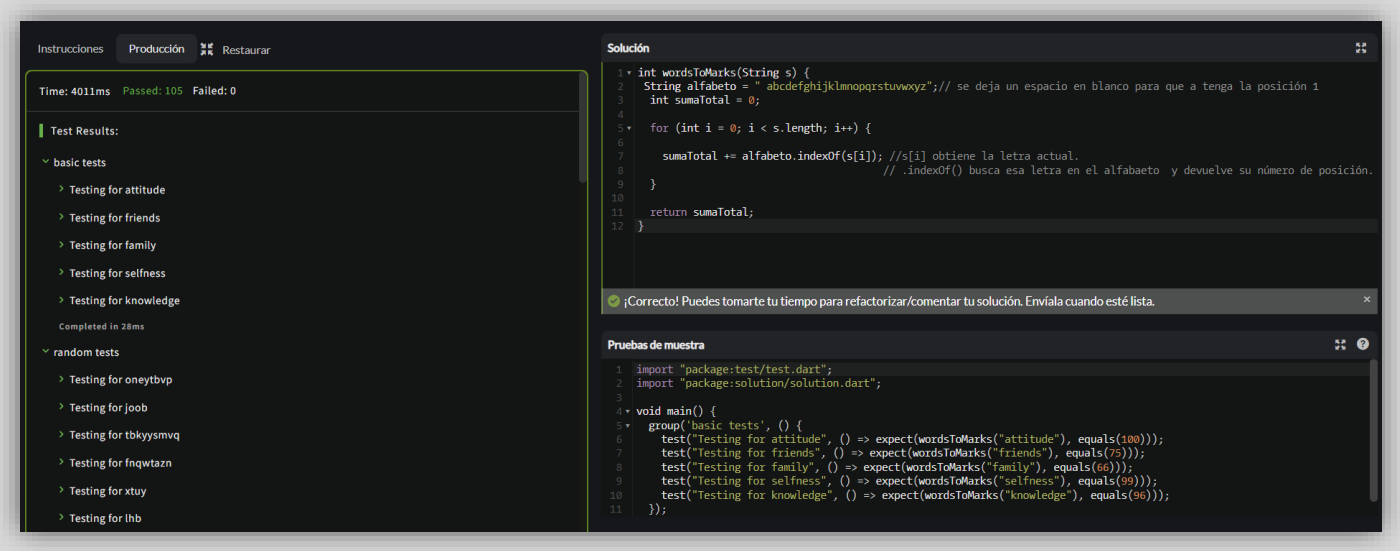
even
odd
2 even
```

kata_12.dart




```dart
int wordsToMarks(String s) {
  String alfabeto = " abcdefghijklmnopqrstuvwxyz";// se deja un espacio en blanco para que a tenga la posición 1
  int sumaTotal = 0;

 for (int i = 0; i < s.length; i++) {
    sumaTotal += alfabeto.indexOf(s[i]); //s[i] obtiene la letra actual.
                                         // .indexOf() busca esa letra en el alfabaeto  y devuelve su número de posición.
 }
 return sumaTotal;
}
```



kata_13.dart

7 kyu Número automórfico (Serie de números especiales n.º 6)

Un número se denomina **automorfo** si y solo si su cuadrado termina en los mismos dígitos que el propio número. Por ejemplo, `25` es un número automorfo porque su cuadrado (`625`) termina en `25` .

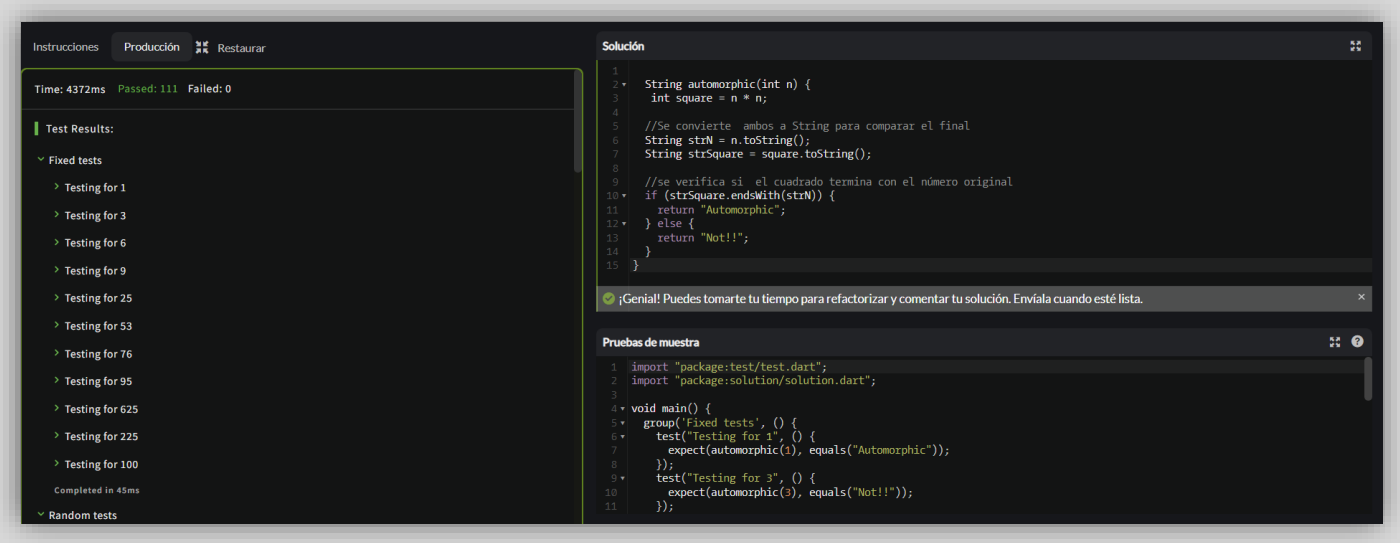
Tarea

Dado un número positivo, determine si es automórfico o no. Si lo es, devuelva "Automorphic"; de lo contrario, devuelva"Not!!"

Ejemplos

- 25 es un número automórfico, porque $25^2 = 625$ termina con 25, entonces regresa "Automorphic".
- 13 no es un número automórfico, porque $13^2 = 169$ no termina con 13, por lo que regresa "Not!!".
- 76 es un número automórfico, porque $76^2 = 5776$ termina con 76, entonces regresa "Automorphic".
- 225 no es un número automórfico, porque $225^2 = 50625$ no termina con 225, regresa "Not!!".
- 625 es un número automórfico, porque $625^2 = 390625$ termina con 625, entonces regresa "Automorphic".
- 1 es un número automórfico, porque $1^2 = 1$ termina con 1, entonces regresa "Automorphic".
- 6 es un número automórfico, porque $6^2 = 36$ termina con 6, entonces regresa "Automorphic".

U07 - Proyecto entregable 02 - Codificando con Dart -

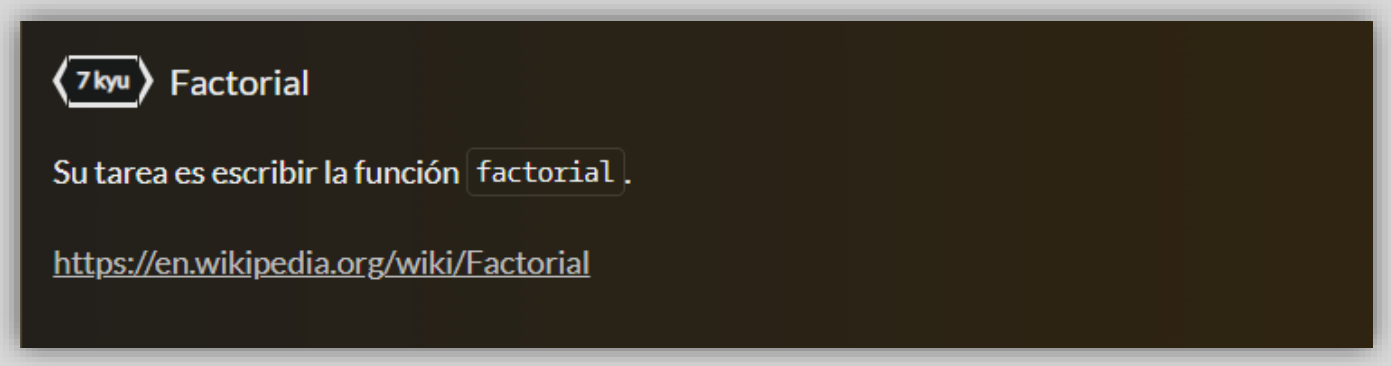


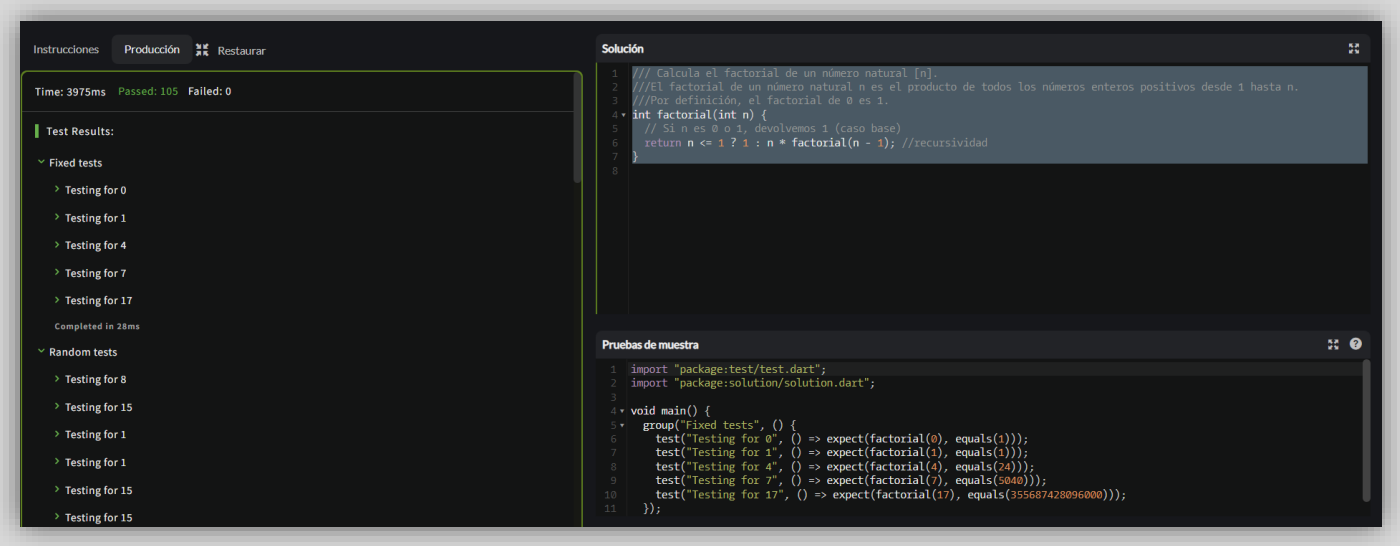

```dart
String automorphic(int n) {
  int square = n * n;

  //Se convierte  ambos a String para comparar el final
  String strN = n.toString();
  String strSquare = square.toString();

  //se verifica si  el cuadrado termina con el número original
  if (strSquare.endsWith(strN)) {
    return "Automorphic";
  } else {
    return "Not!!";
  }
}
```

kata_14.dart





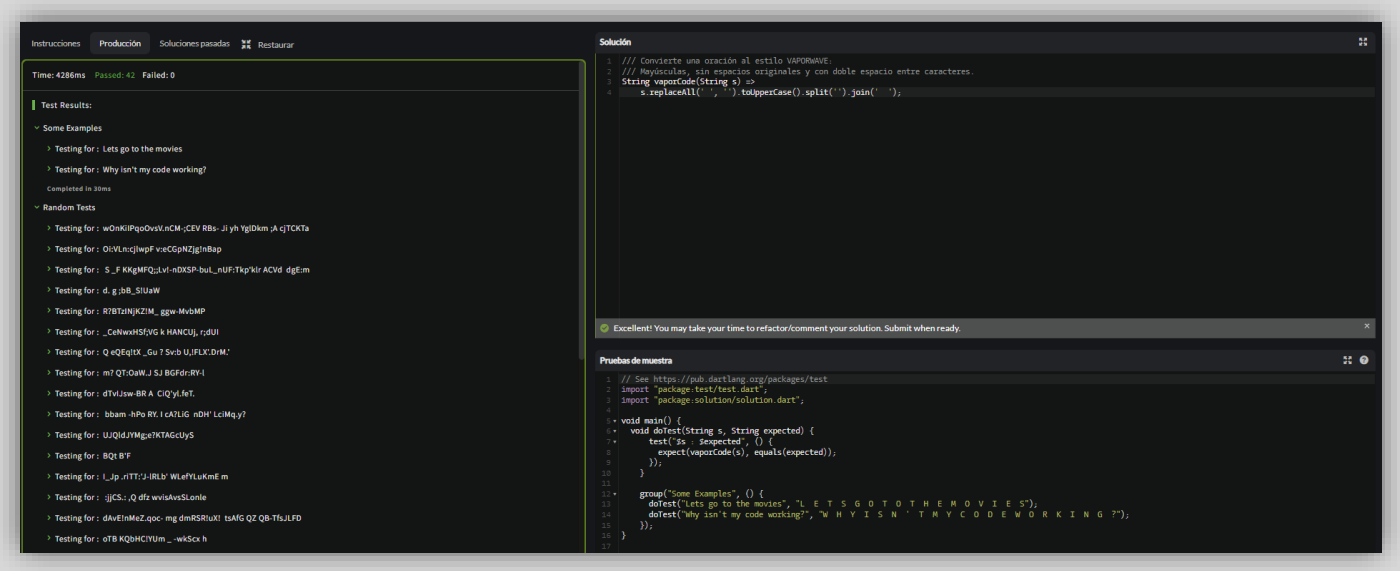

```dart
/// Calcula el factorial de un número natural [n].
///El factorial de un número natural n es el producto de todos los números enteros positivos desde 1 hasta n.
///Por definición, el factorial de 0 es 1.
int factorial(int n) {
  // Si n es 0 o 1, devolvemos 1 (caso base)
  return n <= 1 ? 1 : n * factorial(n - 1); //recursividad
}
```

kata_15.dart

7 kyu CÓDIGO DE VAPOR

Desafío 4 de la semana 1 de ASC (nivel medio n.º 1)

Escriba una función que convierta cualquier oración en una oración VAPORWAVE. Una oración VAPORWAVE convierte todas las letras en mayúsculas y agrega 2 espacios entre cada letra (o carácter especial) para crear este efecto VAPORWAVE.




```dart
/// Convierte una oración al estilo VAPORWAVE:
/// Mayúsculas, sin espacios originales y con doble espacio entre caracteres.
String vaporCode(String s) =>
  s.replaceAll(' ', '').toUpperCase().split('').join('  ');
```

BIBLIOGRAFÍA

Apuntes del Profesor.

https://dart.dev/language?lang=es

https://imaginaformacion.com/tutoriales/aprende-dart-tutorial-de-primeros-pasos

https://es.wikipedia.org/wiki/Dart

https://www.youtube.com/watch?v=5tTDztEQzQQ&t=1s

https://www.youtube.com/watch?v=AOKjGeRv8rk&list=PL2Z95CSZ1N4G9RxzDNWai6krN1dmJqiNS

https://www.youtube.com/watch?v=GsLsLMgehIY&list=PLI_hIu4u7P65OQk_zAxogUjP4YJLQQT1W

https://www.youtube.com/watch?v=zNmDOXbTugE&list=PLCKuOXG0bPi0sIn-nDsi7ma9OV6MEMkxj